

DEVELOPMENT OF AI RESPONSE VALIDATION SYSTEM WITH HALLUCINATION DETECTION ASSISTANCE

Final Project Report

**Infosys Springboard Internship Program
Batch 3 (2026–27)**



Submitted By :-

Name: Agrasha Patel
Program: B.Tech – AIML
Semester: 6th Semester

Submitted To :-

Mentor: Devendra Pratap
Organization: Infosys
Springboard

Declaration (by the students)

I hereby declare that the project entitled: **“Development of AI Response Validation System with Hallucination Detection Assistance”** submitted as part of the **Infosys Springboard Internship Program, Batch 3 (2026–27)**, is my original work carried out under the guidance of the assigned mentor. The project has been developed to study and implement an automated framework for evaluating AI-generated responses based on **relevance, accuracy, hallucination, and completeness**. The system further provides weighted scoring, final verdict generation, batch evaluation, dashboard-based analysis, and automated PDF reporting. I further declare that the work presented in this report has been prepared for academic and internship purposes and has not been submitted elsewhere for the award of any other degree, diploma, certification, or qualification. All external technologies, libraries, datasets, research concepts, and references used during the development of the project have been appropriately acknowledged wherever applicable.

Agrasha Patel

Acknowledgement

I would like to express my sincere gratitude to Infosys Springboard for providing me with the opportunity to work on this project and gain practical exposure to Artificial Intelligence, Large Language Models, and AI evaluation techniques.

I am deeply thankful to my project mentor for providing continuous guidance, valuable feedback, and technical support throughout the development of the AI Response Validation System with Hallucination Detection Assistance.

I also extend my gratitude to my college faculty members and department for their encouragement and support during the internship and project development.

I would like to thank everyone who directly or indirectly contributed to the successful completion of this project. The experience gained through this work has strengthened my understanding of Python development, LLM-based evaluation, RAG concepts, structured outputs, batch processing, testing, dashboard development, and technical documentation.

Finally, I am grateful to my family and friends for their constant encouragement and motivation throughout this journey.

Thank You

Agrasha Patel

Abstract

Large Language Models can generate highly useful responses, but their outputs may contain irrelevant, inaccurate, incomplete, or unsupported information. This project presents an **AI Response Validation System with Hallucination Detection Assistance** designed to automatically evaluate AI-generated responses. The system uses multiple evaluation agents to assess **Relevance, Accuracy, Hallucination, and Completeness**. A weighted scoring mechanism combines these dimensions to generate an overall quality score and final verdict of **Pass, Needs Improvement, or Fail**. The platform supports both **single-response and batch evaluation** using CSV inputs. Retrieval-based validation provides supporting context when reference information is available. The system also includes a **Streamlit dashboard** for visual analysis and a **ReportLab-based PDF reporting module** for structured evaluation reports. Automated end-to-end testing validates the major workflows, scoring logic, batch processing, dashboard-related calculations, and PDF generation. The final system provides a structured approach for evaluating AI responses and identifying potential hallucinations, while also supporting comparison and analysis of responses generated by different AI systems. **Keywords:** Artificial Intelligence, Large Language Models, LLM Evaluation, Hallucination Detection, Relevance, Accuracy, Completeness, RAG, Batch Evaluation, Streamlit.

Table Of Content

Chapter Title :-
Chapter 1 Introduction
Chapter 2: Literature Review
Chapter 3: Existing and Proposed System
Chapter 4: System Design
Chapter 5: Technology Stack
Chapter 6: Implementation
Chapter 7: Single Response Evaluation
Chapter 8: Batch Evaluation
Chapter 9 — Evaluation Scoring Dashboard
Chapter 10 — Evaluation Report Export
Chapter 11 — Error Handling and Reliability
Chapter 12 — Evaluation Methodology
Chapter 13 — Testing and Validation
Chapter 14 — Final Two-AI-System Demonstration
Chapter 15 — Results and Discussion
Chapter 16 — Limitations
Chapter 17 — Future Scope
Chapter 18 — Conclusion
References

CHAPTER 1

Introduction

1.1 Background

Artificial Intelligence has rapidly developed into a technology capable of understanding and generating natural language. Large Language Models can answer questions, summarize information, generate code, and support decision-making. However, generated responses may contain inaccurate facts, irrelevant information, incomplete explanations, or unsupported claims. Therefore, evaluating AI-generated responses systematically is important for determining their reliability, usefulness, and quality before they are used in practical applications.

1.2 Motivation

The motivation for this project comes from the increasing use of AI-generated content and the difficulty of manually checking large numbers of responses. A response may appear fluent while containing factual errors or unsupported information. The project aims to reduce repetitive manual evaluation by providing an automated framework that examines relevance, accuracy, hallucination, and completeness while also providing structured reasoning and an overall evaluation.

1.3 Problem Statement

Large Language Models can generate responses that are convincing but may contain factual inaccuracies, hallucinations, irrelevant information, or missing details. Existing evaluation approaches may require manual assessment or separate tools for different quality dimensions. The project therefore addresses the problem of developing an integrated system capable of automatically evaluating AI responses, detecting unsupported claims, generating consistent

scores, processing batches, visualizing results, and producing structured evaluation reports.

1.4 Need for Proposed System

A systematic response-validation platform is required because manual evaluation becomes difficult as the number of AI-generated responses increases. Different evaluators may also produce inconsistent judgments. The proposed system provides a common evaluation methodology for relevance, accuracy, hallucination, and completeness. It additionally supports batch processing, dashboard visualization, and PDF reporting, allowing users to analyze both individual responses and larger collections of AI-generated outputs efficiently.

1.5 Overview of Proposed Solution

The proposed solution is a multi-agent AI response evaluation platform. A submitted response is analyzed by specialized agents responsible for relevance, accuracy, hallucination detection, and completeness. Their structured results are passed to a Verdict Agent, which calculates a weighted overall score and assigns a final verdict. Optional reference answers and source context provide additional evidence for accuracy and hallucination-related evaluation.

1.6 Project Overview

The system is implemented using Python, Google Gemini API, Streamlit, Pydantic, Pandas, Scikit-learn, and ReportLab. It supports both single-response and CSV-based batch evaluation. The current retrieval mechanism uses TF-IDF through Scikit-learn. Evaluation results are displayed through an interactive dashboard and can be exported as PDF reports containing scores, reasoning, flagged claims, and recommendations.

1.7 Significance of Project

The project demonstrates a practical approach for validating AI-generated responses using multiple evaluation dimensions instead of relying on a single quality measure. It combines language-model evaluation, structured data validation, retrieval support, deterministic scoring, batch processing,

visualization, and report generation. The modular architecture also provides a foundation for future improvements such as semantic retrieval, persistent historical analytics, human review, and additional evaluation dimensions.

1.8 Chapter Summary

This chapter introduced the background, motivation, problem statement, objectives, and significance of the AI Response Validation System. It established the need for automated and structured evaluation of AI-generated content. The following chapters discuss related research, existing approaches, proposed architecture, implementation technologies, evaluation methodology, testing procedures, final demonstration, results, limitations, and possible future improvements.

CHAPTER 2

Literature Review

2.1 Introduction

The evaluation of Large Language Models has become an important research area because generated responses can be fluent while still being incorrect or unsupported. Existing research examines factuality, relevance, groundedness, retrieval quality, and other characteristics of generated text. This chapter reviews the concepts and evaluation approaches relevant to the proposed system, including hallucination detection, Retrieval-Augmented Generation, benchmark datasets, structured outputs, RAGAS, and TruLens.

2.2 Large Language Models

Large Language Models are machine-learning models trained on large amounts of textual data to understand and generate natural language. They are used for question answering, summarization, translation, coding, and conversational applications. Although LLMs can generate highly coherent responses, their outputs are not automatically guaranteed to be factually correct. Therefore, evaluation mechanisms are necessary to determine whether generated responses satisfy specific quality requirements.

2.3 Human Evaluation

Human evaluation involves reviewers manually examining AI-generated responses and judging qualities such as correctness, relevance, completeness, and clarity. Human reviewers can understand context and identify subtle problems, but manual evaluation requires significant time and effort. Results may also vary between evaluators. The proposed system aims to automate repeatable evaluation tasks while retaining structured reasoning and evidence that can support further human review.

2.4 Reference-Based Evaluation

Reference-based evaluation compares an AI-generated response with a trusted reference answer or source. This approach is useful when reliable answers are available. However, different valid answers may use different wording or explanations, making simple text matching insufficient. The proposed system therefore considers semantic evaluation and available evidence rather than depending exclusively on exact textual similarity with a predefined reference answer.

2.5 Relevance Evaluation

Relevance evaluation determines whether an AI-generated response directly addresses the question or instruction provided by the user. A response can contain correct information but still be considered insufficiently relevant if it does not answer the requested topic. The proposed Relevance Judge evaluates this relationship using a five-point scale and provides reasoning explaining why the response received its particular relevance score.

2.6 Accuracy Evaluation

Accuracy evaluation determines whether the information contained in an AI-generated response is factually correct. The Accuracy Judge compares the response with available reference information and source context. It generates a score and reasoning describing the observed correctness. Accuracy evaluation is particularly important because an AI-generated answer can appear confident and well-written while still containing factual errors or contradictions.

2.7 Hallucination Detection

Hallucination refers to information generated by an AI system that is unsupported by available evidence or conflicts with reliable information. The proposed system focuses on identifying unsupported and contradicted claims. The Hallucination Detection Agent records flagged claims, their status, reasoning, and available evidence. This provides more transparency than simply assigning a single hallucination indicator without identifying the problematic content.

2.8 Retrieval-Augmented Generation

Retrieval-Augmented Generation combines information retrieval with language-model processing by providing external information as context. In this project, retrieval supports evidence-based evaluation when additional source information is required. The current implementation uses TF-IDF through Scikit-learn. This should be distinguished from embedding-based retrieval because the current implementation does not use a dedicated semantic vector database.

2.9 Benchmark Datasets

Benchmark datasets provide standardized examples that can be used to study AI performance. TruthfulQA focuses on truthful responses to questions that may encourage common misconceptions, while SQuAD provides question-answering examples based on passages. These datasets are relevant to the project's research because they provide examples of factuality, question answering, and evidence-based evaluation that can support representative testing.

2.10 RAGAS

RAGAS is an evaluation framework associated with Retrieval-Augmented Generation applications. It provides concepts and metrics for examining properties such as answer relevance and faithfulness to retrieved information. These concepts are relevant to the proposed project because the system also evaluates responses using multiple dimensions and supporting evidence. However, the proposed project uses its own evaluation agents and scoring methodology rather than implementing RAGAS directly.

2.11 TruLens

TruLens is an evaluation and tracking framework for AI applications, including retrieval-augmented systems. It provides concepts related to relevance, groundedness, and application-level evaluation. These ideas are relevant to the project because the proposed system also examines whether responses address

questions and whether claims are supported by available context. The project, however, implements its own modular agents and scoring rules.

2.12 Structured Outputs

Structured outputs ensure that AI evaluation results follow a predefined format. The proposed system uses Pydantic models to define evaluation fields, score values, flagged claims, completeness categories, and verdict information. Structured results make it easier to process outputs programmatically and reduce parsing difficulties. They also provide consistency between individual evaluation agents, batch processing, dashboard calculations, and PDF report generation.

2.13 Research Gap

Existing evaluation approaches provide useful techniques for assessing individual response characteristics, but practical validation also requires integrated workflows. The proposed project combines multiple evaluation dimensions with structured outputs, deterministic scoring, batch processing, visualization, and reporting. This integration provides a complete application workflow for examining AI-generated responses rather than treating each evaluation technique as an isolated process.

2.14 Chapter Summary

The literature review established the conceptual foundation for evaluating AI-generated responses. It examined LLMs, human evaluation, reference-based evaluation, relevance, accuracy, hallucination detection, retrieval, benchmark datasets, and established evaluation frameworks. These concepts influenced the design of the proposed multi-agent validation architecture and provided the basis for the evaluation methodology discussed in later chapters.

CHAPTER 3

Existing And Proposed System

3.1 Existing System

Existing AI response evaluation methods commonly involve manual review, reference comparison, text similarity, model-based judging, or specialized evaluation frameworks. These approaches can evaluate specific aspects of response quality but may require separate processes for different dimensions. Large-scale evaluation can therefore become time-consuming and difficult to manage consistently, particularly when dashboard analytics and formal reporting are also required.

3.2 Problems in Existing System

The main limitations include manual effort, inconsistent scoring, limited scalability, difficulty identifying unsupported claims, and separation between evaluation and reporting tools. A response can also be relevant but inaccurate, or accurate but incomplete. Therefore, evaluating only one characteristic may provide an incomplete understanding of overall response quality. These limitations motivate the development of an integrated multi-dimensional evaluation platform.

3.3 Proposed System

The proposed system integrates input handling, optional evidence retrieval, four evaluation agents, deterministic verdict generation, batch processing, dashboard visualization, and PDF reporting. The Relevance, Accuracy, Hallucination, and Completeness agents independently evaluate different response characteristics. Their results are combined by the Verdict Agent using predefined weights and threshold rules to produce a structured overall evaluation.

3.4 Functional Requirements

The system must accept questions and AI-generated responses and optionally support reference answers and source context. It must evaluate relevance, accuracy, hallucination, and completeness and generate an overall verdict. Additional requirements include CSV batch processing, aggregate statistics, dashboard visualization, PDF report export, structured results, input validation, and automated testing of important evaluation workflows.

3.5 Non-Functional Requirements

The system should provide usability, maintainability, reliability, consistency, and reasonable scalability. Evaluation results should follow structured schemas, while sensitive API credentials should remain outside the source code. Batch failures should be handled without unnecessarily terminating successful records. The architecture should also allow individual evaluation agents and supporting modules to be modified or improved without rewriting the complete application.

3.6 Proposed Workflow

The workflow begins when the user provides a question and AI response. Optional reference information or source context is then prepared. The evaluation agents independently assess the response, after which the Verdict Agent combines the scores. In batch mode, multiple records are processed and aggregated. Results are finally presented through the application dashboard and can be exported as PDF documentation.

3.7 Advantages of Proposed System

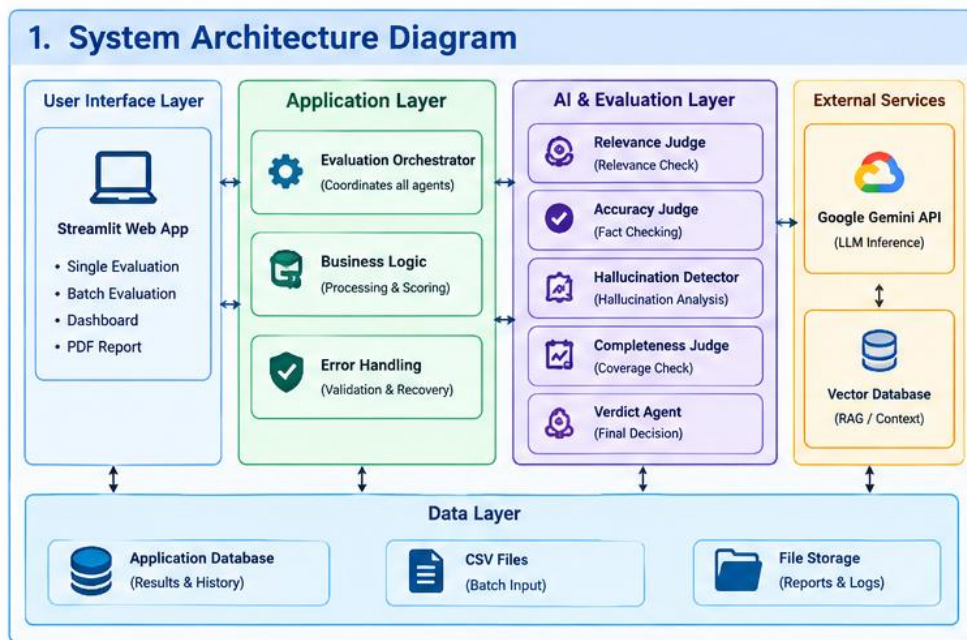
The proposed system provides automated multi-dimensional evaluation, structured reasoning, hallucination assistance, deterministic score aggregation, batch processing, visual analytics, and PDF reporting. Its modular design allows individual components to be tested independently. The platform also provides row-level information, enabling users to investigate individual evaluation results instead of relying only on an aggregate score.

CHAPTER 4

System Design

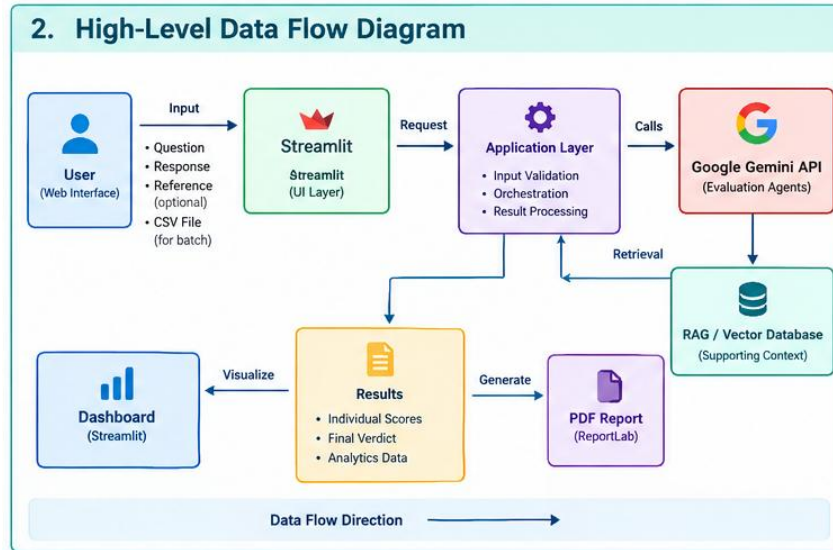
4.1 System Architecture

The system follows a modular architecture consisting of input handling, evidence retrieval, evaluation agents, verdict generation, batch processing, dashboard visualization, and PDF reporting. Each component performs a specific responsibility. This separation improves maintainability and allows individual modules to be tested independently while still supporting a complete end-to-end evaluation workflow.



4.2 High-Level Data Flow

The data flow begins with question and AI-response input. The system validates the information and prepares optional reference answers or source context. Evaluation agents then generate structured results. The Verdict Agent aggregates these results using predefined scoring rules. In batch mode, the results are summarized and passed to dashboard and reporting components for visualization and documentation.

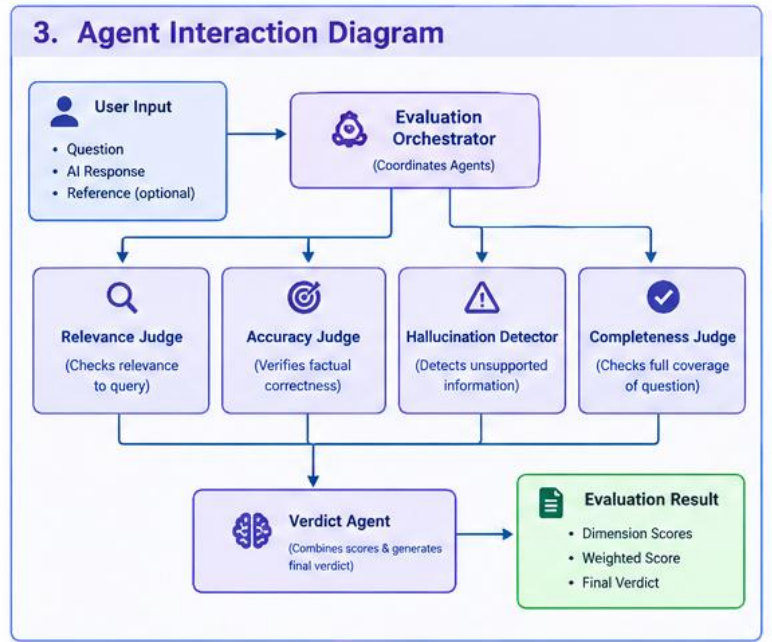


4.3 Component Design

The major components include Input Handler, Retriever, Relevance Judge, Accuracy Judge, Hallucination Detector, Completeness Judge, Verdict Agent, Batch Evaluator, Dashboard, and PDF Reporter. Each component has a clearly defined responsibility. This modular organization allows the system to separate data preparation, evaluation, scoring, analytics, and reporting rather than placing all functionality inside one application module.

4.4 Agent Interaction

The four evaluation agents operate as specialized components. Relevance evaluates question-response alignment, Accuracy examines factual correctness, Hallucination Detection checks source support, and Completeness examines coverage of important requirements. Their outputs are represented using structured models. The Verdict Agent receives these results and applies deterministic scoring rules rather than performing another independent language-model evaluation.

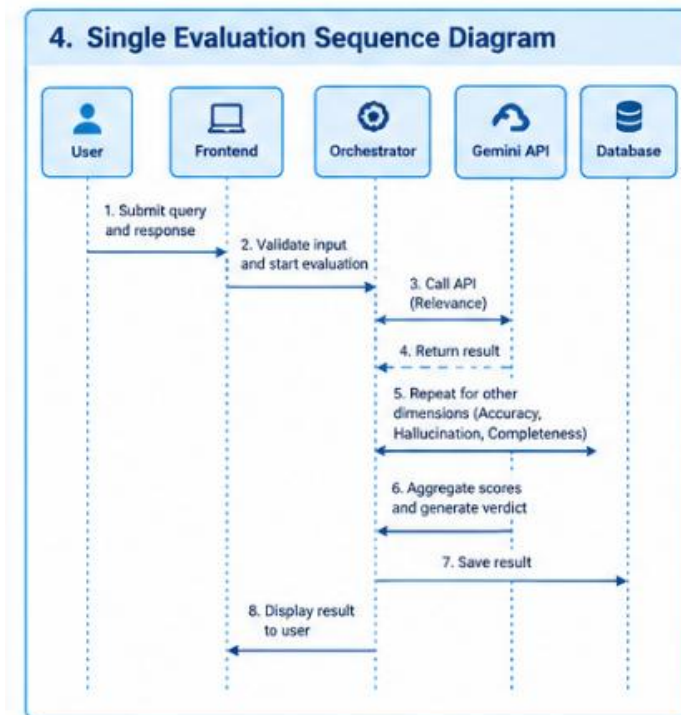


4.5 Data Model

The evaluation data model contains input information, individual agent results, and final verdict information. Input fields include question and AI response, with reference answer and source context as optional fields. Evaluation results contain dimension scores and reasoning, while hallucination results contain flagged claims. Verdict data includes overall score, verdict category, major issues, and reasoning.

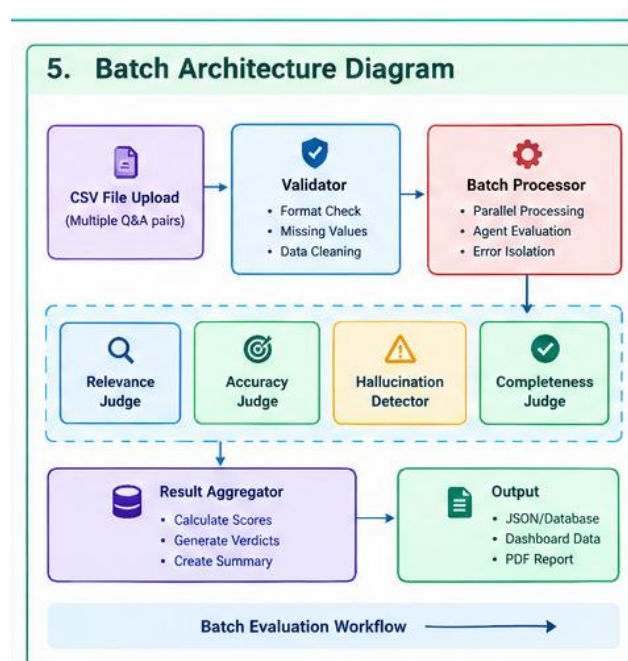
4.6 Single Evaluation Sequence

The single evaluation sequence begins with validated user input and proceeds through the evaluation agents. Each agent returns a structured result, which is passed to the Verdict Agent. The final result contains individual scores, overall score, verdict, and reasoning. This workflow provides a complete assessment of one response and can also be used for demonstration or debugging.



4.7 Batch Architecture

The batch architecture accepts a CSV file containing question-response records. Required columns are validated before evaluation. Records are processed through the evaluation workflow, and row-level results are stored for aggregation. Summary statistics are then calculated and displayed on the dashboard. The same results are used to generate the batch PDF report.



4.8 Security and Configuration

The project uses environment configuration for the Gemini API key. Sensitive credentials should not be placed directly in source files or committed to GitHub. The .gitignore configuration excludes environment files and temporary artifacts. This approach separates application configuration from source code and reduces the risk of accidentally exposing API credentials.

4.9 Chapter Summary

This chapter described the proposed system architecture, data flow, major components, agent interaction, data model, single-response sequence, batch architecture, and security considerations. The modular design ensures that evaluation responsibilities remain separated while allowing the complete system to operate as one integrated response-validation platform.

CHAPTER 5

Technology Stack

5.1 Overview

The project combines programming, artificial intelligence, data processing, retrieval, testing, interface development, and document-generation technologies. Python serves as the primary development language. Google Gemini provides model-based evaluation, while Streamlit provides the application interface. Pydantic supports structured data validation, Pandas manages tabular information, Scikit-learn supports retrieval, and ReportLab generates PDF reports.

5.2 Python

Python is the primary programming language used throughout the project. It provides libraries for artificial intelligence, data processing, testing, web interfaces, retrieval, and document generation. The project uses Python modules to separate agents, orchestration, retrieval, batch processing, scoring, reporting, and testing, making the application easier to maintain and extend.

5.3 Google Gemini API

The Google Gemini API provides the language-model capability required by the evaluation agents. The project uses structured generation so that model responses follow predefined schemas. A centralized LLM client manages API configuration and generation, avoiding duplicated connection logic across agents. The configured API key is loaded through environment variables rather than being hard-coded.

5.4 Pydantic

Pydantic is used to define structured evaluation-result models. These models specify expected fields and valid score values and provide validation for generated results. Pydantic also makes the evaluation data easier to pass

between components because each result follows a predictable structure. This supports reliable processing by the orchestrator, batch evaluator, dashboard, and reporting modules.

5.5 Scikit-learn

Scikit-learn is used for the current TF-IDF-based retrieval component. TF-IDF represents text according to term frequency and inverse document frequency, allowing the retriever to identify documents containing terms relevant to a query. The implementation provides lightweight retrieval support without requiring a separate embedding model or vector database.

5.6 Pandas

Pandas is used for handling CSV files and tabular evaluation results. It supports loading datasets, checking required columns, processing rows, calculating aggregate statistics, and displaying structured batch results. Its DataFrame structure provides a convenient intermediate representation between uploaded CSV data, evaluation outputs, dashboard calculations, and reporting components.

5.7 Streamlit

Streamlit provides the interactive application interface. It supports single-response input, CSV uploading, evaluation-result display, dashboard metrics, charts, and report downloads. The framework allows the Python-based evaluation pipeline to be exposed through a browser interface without requiring a separate frontend framework.

Deploy

Evaluation Mode

Choose evaluation mode:

☒ Single Evaluation

☐ Batch CSV Evaluation

AI Response Validation System

AI-generated response evaluation using Relevance, Accuracy, Hallucination Detection, Completeness and Weighted Verdict.

Single Response Evaluation

Question

Enter the user's question...

AI Response

Enter the AI-generated response...

Reference Answer

Optional reference answer...

5.8 ReportLab

ReportLab is used to generate PDF evaluation reports. The reporting module creates structured documents containing summary statistics, dimension scores, verdict information, flagged claims, and recommendations. The generated PDF is returned to Streamlit so that users can download the completed evaluation report directly from the application.

5.9 Pytest

Pytest is used for automated testing of the project. Tests verify important agent, scoring, batch, PDF, and end-to-end behaviors. During Milestone 4 testing, the end-to-end test suite achieved 8 passing tests out of 8, with an observed execution time of 1.83 seconds.

```
(venv) PS C:\AI-Response-Validation-System> python -m pytest -q tests/test_m4_end_to_end.py
.....
8 passed in 1.83s
```

5.10 Development Environment

Development is performed in a Python virtual environment with project dependencies specified in requirements.txt. The environment contains the required AI, data-processing, testing, dashboard, retrieval, and reporting

libraries. API credentials are maintained separately through environment configuration.

```
(venv) PS C:\AI-Response-Validation-System> python -m streamlit run app.py
2026-10-04 13:25:35.284 Uvicorn server started on :::8501

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.236.177:8501
```

5.11 Chapter Summary

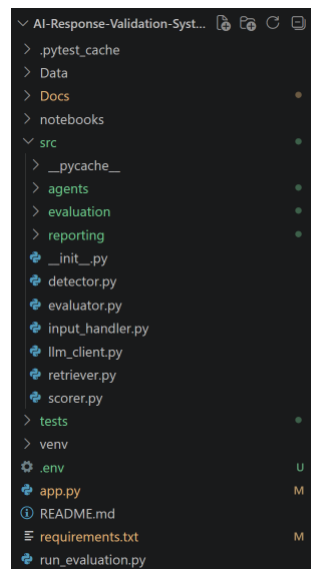
This chapter described the technologies used to build the project and explained their individual roles. The selected technology stack provides the capabilities required for language-model evaluation, structured data handling, retrieval, batch processing, interface development, testing, visualization, and PDF reporting.

CHAPTER 6

Implementation

6.1 Project Structure

The project is organized into separate directories for agents, evaluation, retrieval, input handling, scoring, reporting, tests, documentation, and application code. This modular structure prevents excessive coupling between components. Each module can therefore be developed and tested independently while contributing to the overall response-validation workflow.



6.2 LLM Client

The LLM client loads the Gemini API key from environment configuration and initializes the model client. A common structured-generation function receives an evaluation prompt and response schema. The model is configured for JSON output and controlled generation. Centralizing this functionality simplifies API management and prevents repeated configuration across individual evaluation agents.

6.3 Relevance Judge Implementation

The Relevance Judge receives the question and AI-generated response. Its prompt instructs the model to determine how directly the response addresses the

question and return a score from 1 to 5 with reasoning. The result is represented through a Pydantic model, allowing the application to process the score consistently.

6.4 Accuracy Judge Implementation

The Accuracy Judge evaluates factual correctness using the question, response, reference answer, and source context where available. It produces a structured score and reasoning. The evaluator can distinguish correct, partially correct, incorrect, and contradictory information. This component forms an important part of the final verdict because accuracy receives the highest weight in the scoring model.

6.5 Hallucination Detector Implementation

The Hallucination Detection Agent analyzes claims made within an AI response against available source context. It identifies unsupported and contradicted claims and records their status, reasoning, and evidence. The overall result contains a Boolean hallucination indicator and a collection of flagged claims, allowing users to inspect specific problematic statements.

6.6 Completeness Judge Implementation

The Completeness Judge identifies important requirements or sub-questions within the original question and determines whether they are addressed. It categorizes aspects as addressed, partially addressed, or missing and assigns a score from 1 to 5. This provides a structured method for identifying responses that are relevant but fail to cover all required information.

6.7 Verdict Agent Implementation

The Verdict Agent combines relevance, accuracy, completeness, and hallucination results using predefined weights. Relevance contributes 25%, accuracy 30%, completeness 25%, and hallucination 20%. Hallucination receives a score of 5 when none is detected and 1 when detected. Critical failure rules are applied before the final verdict is returned.

6.8 Orchestrator

The Evaluation Orchestrator coordinates the individual evaluation agents. It normalizes optional reference and source fields, calls each agent, and passes their outputs to the Verdict Agent. The final result contains the question, response, individual evaluation results, and overall verdict. This component therefore acts as the main coordination layer for single-response evaluation.

6.9 Batch Evaluation Agent

The Batch Evaluation Agent processes multiple question-response records and returns structured row-level evaluations. It evaluates relevance, accuracy, hallucination, and completeness for each record. The batch design reduces unnecessary repeated orchestration and provides a common output structure that can subsequently be converted into deterministic verdicts and aggregate statistics.

6.10 Batch Evaluator

The Batch Evaluator validates the uploaded DataFrame, checks required columns, prepares evaluation records, processes results, and calculates batch statistics. It records row-level scores, reasoning, flagged claims, completeness information, overall scores, verdicts, and major issues. Successful and failed records are tracked separately for dashboard and report generation.

6.11 Dashboard Implementation

The Streamlit dashboard presents batch-level evaluation metrics such as total responses, successful and failed evaluations, pass rate, average quality score, dimension averages, hallucination rate, and verdict distribution. Charts provide visual summaries of dimension scores and verdict counts. The current quality trend represents the order of records within the current batch.

6.12 PDF Reporting Implementation

The PDF reporting module receives evaluation statistics and row-level results and creates a structured PDF using ReportLab. The report contains summary information, dimension breakdowns, verdict information, flagged claims, and

recommendations. The generated PDF is provided through a Streamlit download button, completing the reporting workflow.

6.13 Error Handling

The application validates required input fields and CSV columns before evaluation. Structured outputs are validated using Pydantic models. Batch processing records failures where possible rather than terminating all processing. External API problems such as quota exhaustion or service unavailability are reported separately from application-level validation errors.

6.14 Configuration

The application configuration includes the Gemini API key, model configuration, and Python dependencies. The API key is stored in `.env` during development and excluded from version control. The `requirements.txt` file records the libraries needed to reproduce the development environment and execute the project.

6.15 Chapter Summary

This chapter explained the implementation of the major project components, including the LLM client, evaluation agents, orchestrator, batch evaluator, dashboard, reporting system, error handling, and configuration. Together, these components form the complete technical implementation of the AI Response Validation System.

CHAPTER 7

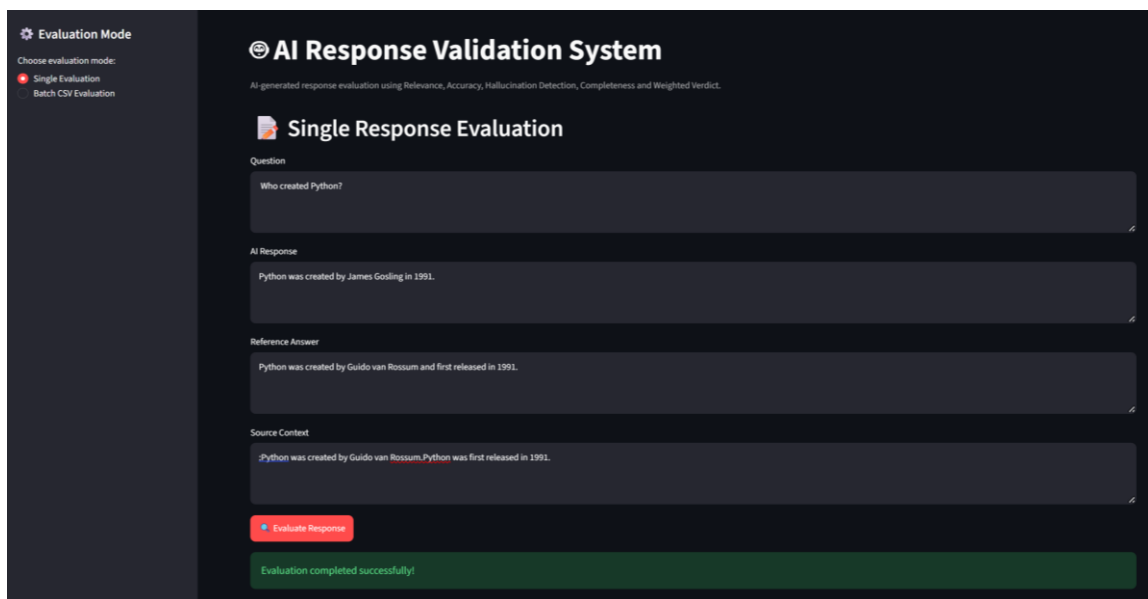
Single Response Validation

7.1 Purpose

Single-response evaluation allows the system to examine one AI-generated response in detail. It is useful for demonstrations, debugging, prompt evaluation, and understanding individual quality issues. The workflow provides dimension-specific scores and reasoning before calculating the overall verdict.

7.2 Input Fields

The single evaluation interface accepts a question and AI-generated response as required inputs. A reference answer and source context can optionally be supplied. Reference information supports factual comparison, while source context supports evidence-based evaluation and hallucination detection.



The screenshot displays the 'AI Response Validation System' interface. On the left, a sidebar titled 'Evaluation Mode' shows 'Single Evaluation' as the selected mode. The main panel, titled 'Single Response Evaluation', contains four text input fields: 'Question' (with the text 'Who created Python?'), 'AI Response' (with the text 'Python was created by James Gosling in 1991.'), 'Reference Answer' (with the text 'Python was created by Guido van Rossum and first released in 1991.'), and 'Source Context' (with the text 'Python was created by Guido van Rossum. Python was first released in 1991.'). Below these fields is a red 'Evaluate Response' button. At the bottom, a green status bar indicates 'Evaluation completed successfully!'.

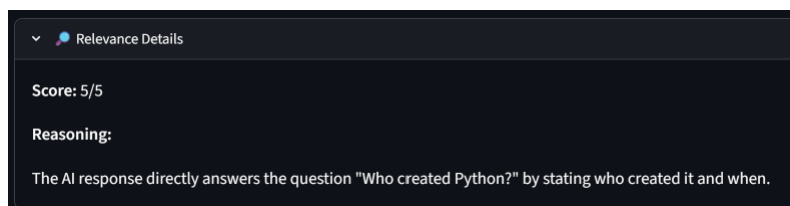
7.3 Evaluation Flow

The submitted question and response are first validated. The system then invokes the Relevance, Accuracy, Hallucination, and Completeness agents. Their structured results are passed to the Verdict Agent, which calculates the

weighted score and final verdict. The complete result is displayed through the Streamlit interface.

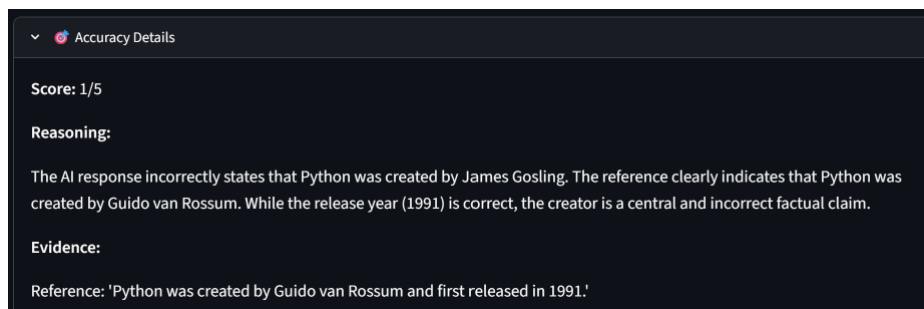
7.4 Relevance Result

The relevance result contains a score between 1 and 5 and supporting reasoning. The score indicates how directly the generated response addresses the original question. This allows users to distinguish between responses that answer the requested topic and responses that contain information but fail to address the actual requirement.



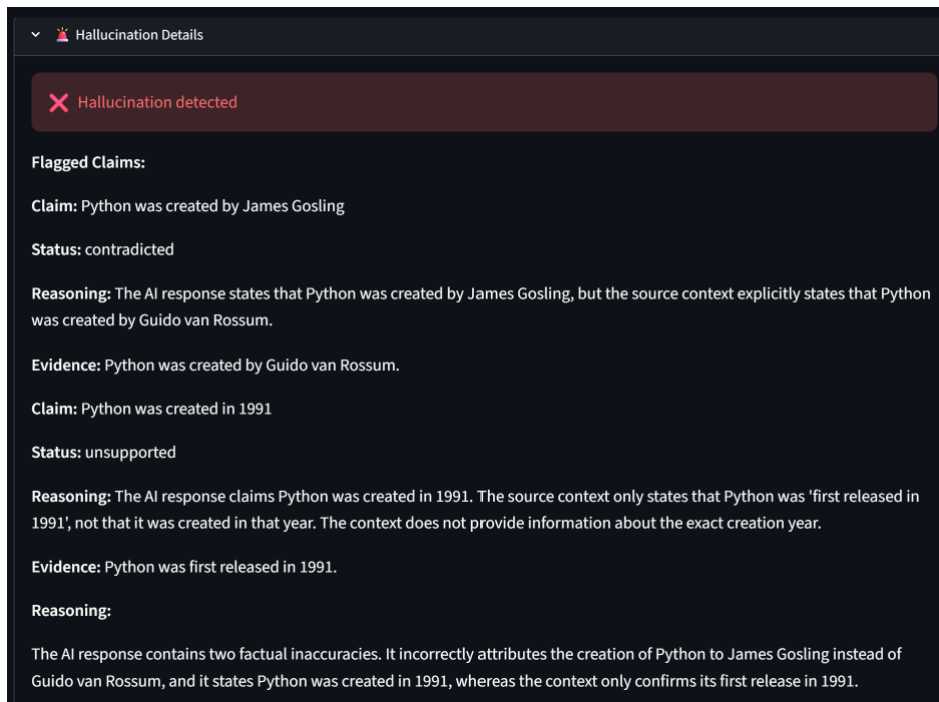
7.5 Accuracy Result

The accuracy result provides a five-point score and reasoning based on available reference information or source context. It identifies whether the response is factually correct, partially correct, or contains significant inaccuracies. The result contributes 30% to the overall weighted score, making accuracy the highest-weighted evaluation dimension.



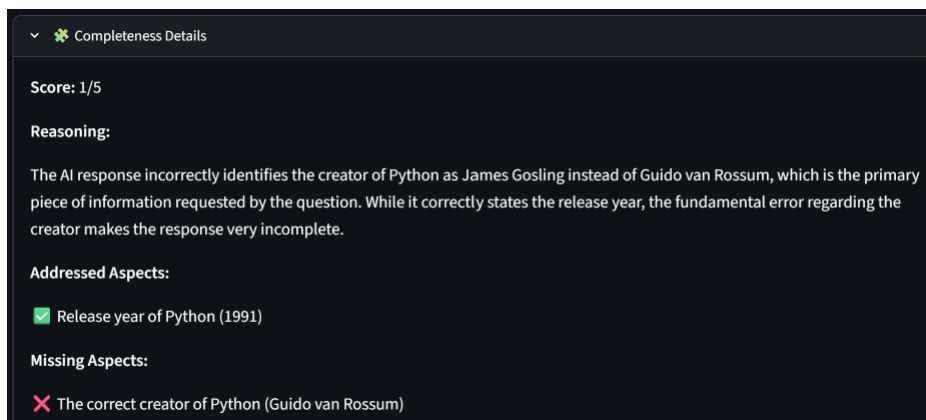
7.6 Hallucination Result

The hallucination result indicates whether unsupported or contradicted claims were detected. When claims are flagged, the system displays their status, reasoning, and evidence. This makes the hallucination assessment more transparent because users can examine the specific statements responsible for the detection.



7.7 Completeness Result

The completeness result identifies which important aspects of the question were addressed, partially addressed, or missing. It provides a score from 1 to 5 and explanatory reasoning. This allows the system to distinguish a response that is relevant but incomplete from one that fully satisfies the requirements of the question.



7.8 Final Verdict

The final result combines all four dimensions into an overall score. The system classifies the response as Pass, Needs Improvement, or Fail according to predefined thresholds. Accuracy score 1 and detected hallucination are treated as critical failure conditions.



Final Verdict

✖ Fail

7.9 Example Evaluation

A representative test can use a factual question with an intentionally incorrect response. For example, a question asking for the capital of France can be paired with an answer stating Berlin. With appropriate reference information, the system can demonstrate how accuracy problems are identified and how the final verdict responds to a serious factual error.

7.10 Chapter Summary

Single-response evaluation demonstrates the complete evaluation pipeline at an individual level. It provides detailed information from each specialized agent and combines these results into an interpretable final verdict, making it useful for testing and demonstrating the core functionality of the proposed system.

CHAPTER 8

Batch Evaluation

8.1 Purpose

Batch evaluation enables the system to process multiple AI-generated responses through a single CSV workflow. It is useful for testing an AI system against a representative dataset and for generating aggregate quality statistics. The batch workflow extends the single-response evaluation process while preserving row-level evaluation information.

8.2 CSV Format

The batch CSV requires two columns: `question` and `ai_response`. Two optional columns, `reference_answer` and `source_context`, can provide additional evidence. Each row represents one evaluation record, allowing multiple questions and AI-generated responses to be processed systematically.

8.3 Batch Validation

Before evaluation begins, the system checks whether required columns are available. Invalid or incomplete input is reported before model evaluation proceeds. This prevents avoidable processing errors and ensures that each evaluation record contains the minimum information required by the system.

Uploaded Data

question	ai_response	reference_answer	source_context
What is Python?	Python is a high-level programming language.	Python is a high-level interpreted programming language.	Python is a high-level interpreted programming language.
What is the capital of France?	The capital of France is Berlin.	The capital of France is Paris.	Paris is the capital of France.

CSV structure is valid — 2 record(s) found.

Processing 2 record(s) using one Gemini request for the complete batch.

Run Batch Evaluation

8.4 Batch Processing

Validated records are prepared for evaluation and processed through the batch evaluation workflow. Each record receives relevance, accuracy, hallucination,

and completeness results. The results are then passed through deterministic verdict generation and stored for aggregate analysis.

8.5 Batch Statistics

The system calculates total responses, successful evaluations, failed evaluations, average quality score, average dimension scores, verdict counts, pass rate, and hallucination frequency. These statistics provide a high-level view of the quality of the evaluated dataset while allowing users to inspect individual records separately.

8.6 Error Isolation

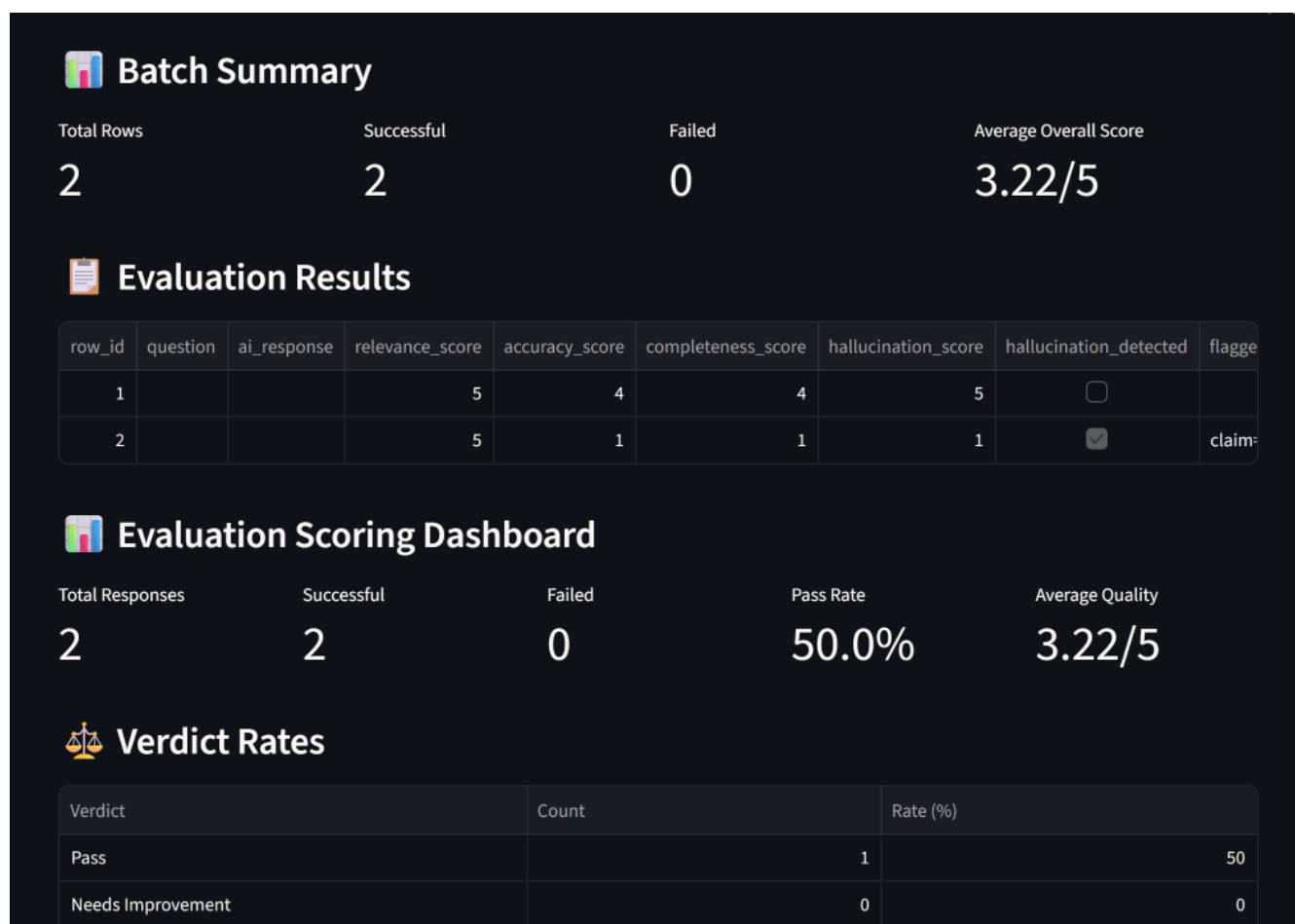
Where possible, an error affecting one record should not unnecessarily terminate the complete batch. Failed records are tracked separately so that successfully processed records remain available for analysis. This approach improves resilience when datasets contain malformed records or individual model-evaluation failures.

8.7 Sample Dataset Design

A useful evaluation dataset should include correct, incorrect, incomplete, irrelevant, and evidence-sensitive responses. Such diversity allows the system to exercise different evaluation dimensions. The final demonstration dataset should use the same questions when comparing two AI systems so that the evaluation conditions remain consistent.

8.8 Batch Evaluation Output

The batch results table displays row-level evaluation information, including dimension scores, hallucination status, overall score, verdict, and relevant reasoning fields. This provides a detailed view before aggregate results are interpreted through the dashboard.



8.9 Chapter Summary

Batch evaluation provides a scalable workflow for examining multiple AI responses. It validates uploaded datasets, evaluates individual records, calculates aggregate statistics, tracks failures, and provides the data required for dashboard visualization and PDF report generation.

CHAPTER 9

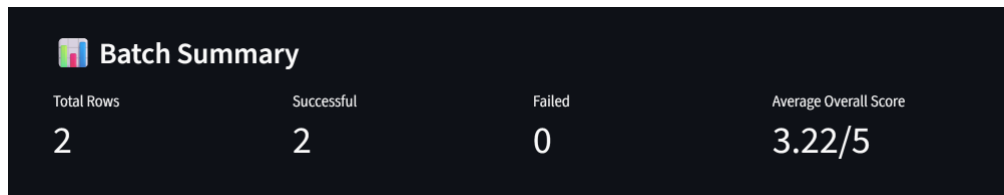
Evaluation Scoring Dashboard

9.1 Purpose

The Evaluation Scoring Dashboard converts batch evaluation results into visual and numerical summaries. It allows users to understand overall response quality without examining every record individually. The dashboard also provides access to detailed results and failed evaluations for deeper analysis.

9.2 Key Performance Indicators

The dashboard displays total responses, successful evaluations, failed evaluations, pass rate, average quality score, verdict rates, average dimension scores, and hallucination rate. These indicators summarize the overall performance of the evaluated batch and provide a starting point for identifying quality-related patterns.

A screenshot of a 'Batch Summary' dashboard. It features a dark blue header with a small icon and the title 'Batch Summary'. Below the header, there are four columns with labels and values: 'Total Rows' with value '2', 'Successful' with value '2', 'Failed' with value '0', and 'Average Overall Score' with value '3.22/5'.

Total Rows	Successful	Failed	Average Overall Score
2	2	0	3.22/5

9.3 Dimension Scores

Average relevance, accuracy, completeness, and hallucination scores are displayed separately. Examining these values independently is important because an overall score can hide differences between evaluation dimensions. The dimension metrics help users identify whether a dataset has problems primarily related to factual accuracy, response coverage, relevance, or hallucination.

9.4 Verdict Distribution

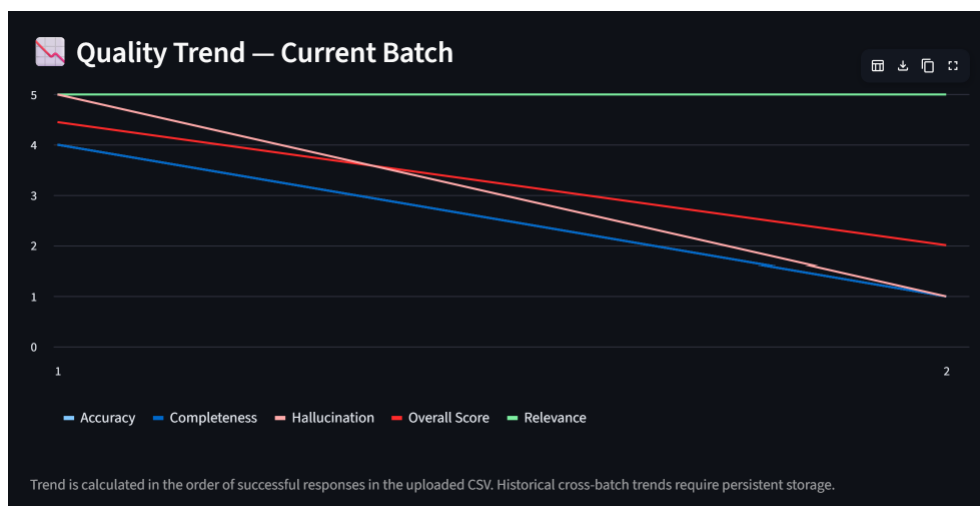
The verdict distribution presents the number or percentage of Pass, Needs Improvement, and Fail evaluations. This provides an immediate overview of the batch. The distribution should be interpreted together with individual evaluation

results because aggregate categories do not explain the specific reasons behind each verdict.



9.5 Quality Trend

The current dashboard provides a quality trend based on the order of responses within the uploaded batch. It shows how overall scores vary across the current dataset. This should not be interpreted as a historical trend across multiple batches because persistent historical storage is not part of the current implementation.



9.6 Hallucination Frequency

Hallucination frequency indicates the proportion of evaluated responses for which the detector identified unsupported or contradicted claims. The metric

should always be interpreted together with source availability because limited source context can affect the ability to verify claims.

9.7 Dashboard Verification

Dashboard calculations should be manually compared with the underlying evaluation table. Counts, averages, rates, and verdict distributions should match the stored results. This verification is important because dashboard correctness is part of the Milestone 4 requirements.

9.8 Chapter Summary

The dashboard provides a visual summary of batch evaluation results. It combines key performance indicators, dimension averages, verdict distribution, hallucination frequency, and quality trends, enabling users to understand the evaluated dataset while retaining access to detailed row-level information.

CHAPTER 10

Evaluation Report Export

10.1 Purpose

The Evaluation Report Export module converts batch evaluation results into a structured PDF document. It provides a permanent and shareable representation of the evaluation process and results. The report can be used for project documentation, technical review, demonstrations, and final submission evidence.

10.2 Batch Report Contents

The PDF report contains evaluation summary information, successful and failed response counts, average dimension scores, verdict distribution, hallucination frequency, individual evaluation results, flagged claims, reasoning, and recommendations. These sections provide both aggregate and detailed views of the evaluated batch.

10.3 Report Generation Process

The application prepares a report payload containing batch statistics and row-level results. The reporting module then uses ReportLab to construct the PDF. After generation, the PDF is returned to the Streamlit application and made available through a download button.



10.4 Accuracy of Export

The generated PDF should accurately reflect the underlying evaluation results. Summary statistics, dimension scores, verdicts, flagged claims, and recommendations should correspond to the batch data displayed by the application. PDF verification is therefore necessary to ensure that exported documentation does not contain inconsistent information.

10.5 Recommendations

The report includes recommendations based on observed evaluation issues. For example, repeated missing aspects may indicate a need for more complete responses, while unsupported claims may indicate a need for stronger evidence grounding. Recommendations should describe observed issues rather than making unsupported assumptions about the underlying AI system.

AI Response Validation System	
Batch Evaluation Report	
Generated: 2026-10-04 13:54:41	
1. Batch Summary	
Metric	Value
Total responses	2
Successful evaluations	2
Failed evaluations	0
Pass rate	50.0%
Needs Improvement	0
Fail	1
Average overall score	3.22/5
Hallucination frequency	1 (50.0%)
2. Dimension Breakdown	
Dimension	Average Score / 5
Relevance	5.00
Accuracy	2.50
Completeness	2.50
Hallucination	3.00
3. Flagged Responses	
Response 1 — Row 2	
Question:	
AI Response:	
Verdict: Fail Overall Score: 2.0/5	
Flagged claims:	
• claim="The capital of France is Berlin." status="contradicted" reasoning="The AI response claims the capital of France is Berlin, but the source context and reference answer clearly state it is Paris." evidence="SOURCE CONTEXT: Paris is the capital of France. REFERENCE ANSWER: The capital of France is Paris."	
Major issues: The response contains significant factual inaccuracies. The response is substantially incomplete. Unsupported or contradicted claims were detected.	

10.6 Chapter Summary

PDF reporting completes the evaluation workflow by transforming application results into a structured document. It allows users to preserve evaluation findings, review flagged responses, and provide formal evidence of the batch evaluation process.

CHAPTER 11

Error Handling And Reliability

11.1 Importance of Error Handling

The application depends on user inputs, CSV files, structured model responses, retrieval, and external API services. Errors may therefore occur at multiple stages. Proper error handling ensures that users receive understandable messages and that recoverable problems do not unnecessarily terminate the complete evaluation process.

11.2 Input Validation Errors

The system validates required fields and batch CSV columns before evaluation. Missing question or ai_response columns are reported as input errors. Validation prevents the system from sending incomplete records to the evaluation pipeline and provides users with an opportunity to correct the dataset.

11.3 Malformed Records

Individual records may contain missing values or invalid information. The batch workflow should identify such records and record their failure status where possible. This approach allows valid records to continue through evaluation while preserving information about problematic rows.

11.4 Structured Output Errors

Evaluation agents are expected to return structured results matching predefined Pydantic schemas. If a generated response does not conform to the required structure, validation can detect the problem. This prevents malformed model outputs from silently propagating into dashboard calculations or final reports.

11.5 API Errors

External Gemini API errors may include temporary service unavailability and quota exhaustion. Temporary failures may sometimes be retried, while resource-

exhaustion errors may require waiting or adjusting quota usage. Such errors should be clearly distinguished from application-code errors.

11.6 Batch Resilience

Batch resilience means that a failure in one record should not unnecessarily invalidate all successful evaluations. The system therefore tracks successful and failed records separately. This design is especially useful when processing larger datasets where occasional problematic records may occur.

11.7 Configuration Errors

The application requires a valid Gemini API key. If the key is missing from environment configuration, the application should report the configuration problem clearly. Credentials should not be embedded directly in source code or committed to the repository.

11.8 Reliability Considerations

Application reliability depends on both internal implementation and external service availability. Structured validation, deterministic scoring, and input checking improve reliability within the application. However, external model availability, API quota, and retrieval quality remain factors outside complete application control.

11.9 Chapter Summary

This chapter described the major error-handling mechanisms and reliability considerations of the project. Proper validation and failure isolation improve the robustness of the application, while external API and retrieval limitations must be considered when interpreting system behavior.

CHAPTER 12

Evaluation Methodology

12.1 Evaluation Dimensions

The system evaluates four primary dimensions: relevance, accuracy, hallucination, and completeness. Each dimension addresses a different aspect of response quality. The individual results are kept separate before aggregation so that users can understand which characteristics contributed to the final evaluation.

12.2 Relevance Scale

Relevance is evaluated using a five-point scale. A score of 5 represents a response that directly addresses the question, while lower scores indicate increasing levels of irrelevance or insufficient focus. The score is accompanied by reasoning so that the evaluation is not represented solely as a numerical value.

12.3 Accuracy Scale

Accuracy is evaluated from 1 to 5. Higher scores represent stronger factual correctness, while lower scores indicate increasing factual problems. The Accuracy Judge uses available reference answers and source context where supplied. An accuracy score of 1 is defined as a critical failure condition.

12.4 Completeness Scale

Completeness is evaluated from 1 to 5. A score of 5 indicates that important requirements are fully addressed. Lower scores indicate increasing levels of missing information. The evaluator also lists addressed, partially addressed, and missing aspects to explain the score.

12.5 Hallucination Score

For weighted aggregation, hallucination is represented as a score of 5 when no hallucination is detected and 1 when hallucination is detected. The detector

separately records the actual flagged claims and their statuses. This allows the numerical score to remain compatible with the weighted model while preserving detailed evidence.

12.6 Weighted Scoring

The overall score uses four predefined weights: relevance 25%, accuracy 30%, completeness 25%, and hallucination 20%. The formula is: Overall Score = $(\text{Relevance} \times 0.25) + (\text{Accuracy} \times 0.30) + (\text{Completeness} \times 0.25) + (\text{Hallucination} \times 0.20)$.

12.7 Verdict Thresholds

The final score is classified into three categories. Scores from 4.00 to 5.00 receive Pass, scores from 2.50 to 3.99 receive Needs Improvement, and scores from 1.00 to 2.49 receive Fail. These thresholds provide a consistent mapping between the weighted score and final evaluation category.

12.8 Critical Failure Rules

Two critical rules override normal weighted classification. A response with an accuracy score of 1 receives a Fail verdict. A response for which hallucination is detected also receives Fail. These rules ensure that severe factual or evidence-related problems cannot be hidden by high scores in other dimensions.

12.9 Consistency Evaluation

Consistency can be examined by evaluating repeated or similar questions and checking whether the system produces logically similar scores and reasoning. Exact wording does not need to be identical. Instead, the focus is whether similar response qualities are treated according to the same scoring criteria.

12.10 Representative Test Cases

Representative test cases include correct, incorrect, incomplete, irrelevant, unsupported, and contradicted responses. Additional cases cover missing reference information, malformed CSV files, and evaluation-service failures. These cases ensure that the system is tested across both normal and problematic conditions.

12.11 Chapter Summary

This chapter defined the evaluation methodology used by the system. It established the scoring scales, weighted formula, verdict thresholds, critical failure conditions, consistency approach, and representative test scenarios used to validate the response-evaluation workflow.

CHAPTER 13

Testing And Validation

13.1 Testing Objectives

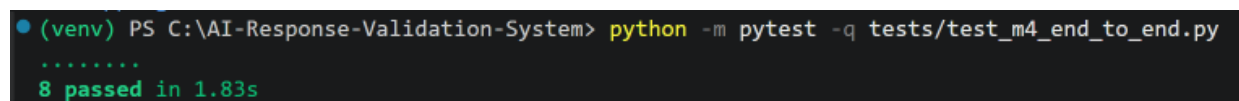
Testing verifies whether the implemented system behaves according to the defined requirements. The project testing process covers individual evaluation components, deterministic scoring, batch processing, PDF generation, and end-to-end workflows. Testing also examines negative cases to determine whether malformed input and external failures are handled appropriately.

13.2 Test Categories

Testing includes agent-level tests, verdict tests, batch evaluation tests, dashboard calculation checks, PDF generation tests, single-response workflow tests, batch workflow tests, and error-handling tests. Combining automated and manual testing provides broader coverage because interface behavior cannot always be verified through backend tests alone.

13.3 M4 End-to-End Test Suite

The Milestone 4 end-to-end test suite achieved 8 passing tests out of 8. The observed execution time was 2.04 seconds, giving a 100% passing rate for the executed suite. These results demonstrate successful execution of the tested M4 workflows.

A terminal window with a dark background. The prompt is '(venv) PS C:\AI-Response-Validation-System>'. The command entered is 'python -m pytest -q tests/test_m4_end_to_end.py'. The output shows '.....' followed by '8 passed in 1.83s' on the next line.

```
(venv) PS C:\AI-Response-Validation-System> python -m pytest -q tests/test_m4_end_to_end.py
.....
8 passed in 1.83s
```

13.4 Test Coverage

The tests cover important interactions between evaluation, batch processing, verdict generation, and PDF reporting. The test design also includes mocked evaluation objects where required. This allows deterministic verification of application behavior without depending on live model responses for every test case.

13.5 Batch Average Verification

Batch average calculations were verified against the implementation. The expected value was aligned with the actual rounding behavior used by the system. This ensures that dashboard and report statistics are consistent with the underlying numerical calculations rather than relying on manually rounded assumptions.

13.6 PDF Verification

PDF generation was tested by verifying that the reporting function produces valid PDF byte content. The current reporting interface returns PDF bytes, so the test checks the output type and PDF signature. This confirms that the generated report can be passed to the application's download mechanism.

13.7 Negative Testing

Negative testing examines malformed input, missing required columns, service failures, unsupported responses, contradictions, and incomplete answers. These cases help verify that the system produces appropriate failure information rather than silently generating misleading results.

13.8 Manual Streamlit Validation

Manual validation is required for the Streamlit interface because automated tests do not fully capture visual layout, interactive controls, charts, and download behavior. The single evaluation, batch upload, dashboard, and PDF download should therefore be manually checked before final demonstration.

13.9 Test Result Summary

The automated M4 suite provides an observed result of 8/8 passing tests in 2.04 seconds. Manual interface verification should complement this result. Final two-AI-system evaluation results should also be recorded separately because they represent demonstration data rather than software-test outcomes.

13.10 Chapter Summary

Testing confirms the behavior of the tested implementation paths and demonstrates successful M4 end-to-end execution. Automated testing is

complemented by manual interface checks and final demonstration testing to provide broader evidence of system functionality.

CHAPTER 14

Final Two-AI-System Demonstration

14.1 Purpose

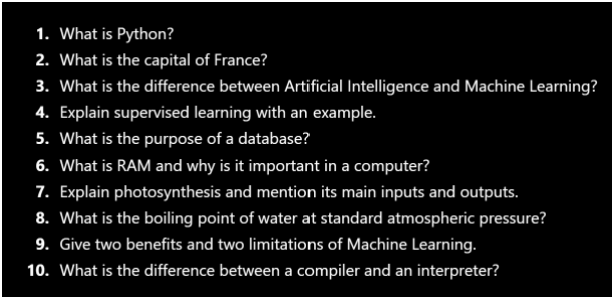
The final demonstration evaluates outputs from two distinct AI systems using a common dataset. The same questions and evaluation methodology should be used for both systems. This demonstrates that the proposed validation platform can process responses from different AI systems and produce structured, comparable evaluation information.

14.2 Selected AI Systems

The planned demonstration uses ChatGPT and Google Gemini as two distinct AI systems. Their responses should be collected using the same question set and evaluated under the same project configuration. The final report should include only actual responses and evaluation results generated during the demonstration.

14.3 Common Dataset

The common dataset contains ten questions covering programming, artificial intelligence, databases, science, computing, and machine learning. The same questions should be submitted to both AI systems. Reference answers and source context should be prepared consistently so that the evaluation conditions remain comparable.

- 
1. What is Python?
 2. What is the capital of France?
 3. What is the difference between Artificial Intelligence and Machine Learning?
 4. Explain supervised learning with an example.
 5. What is the purpose of a database?
 6. What is RAM and why is it important in a computer?
 7. Explain photosynthesis and mention its main inputs and outputs.
 8. What is the boiling point of water at standard atmospheric pressure?
 9. Give two benefits and two limitations of Machine Learning.
 10. What is the difference between a compiler and an interpreter?

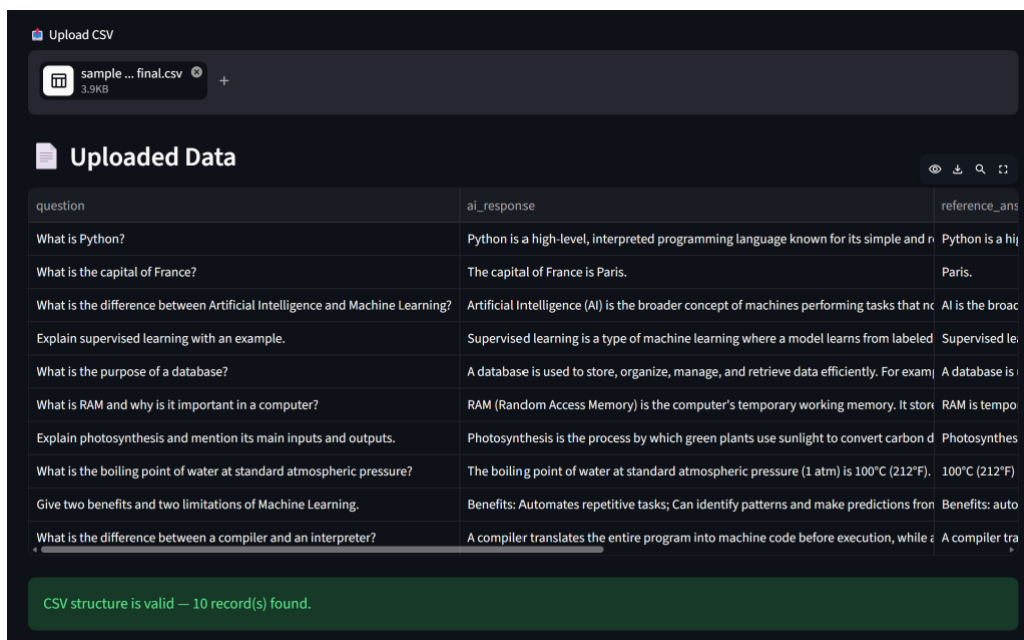
14.4 Dataset Design

The dataset should contain questions capable of producing different response qualities. The demonstration should include opportunities to observe correct,

incomplete, inaccurate, irrelevant, and evidence-sensitive responses. This ensures that the validation system demonstrates more than simple successful evaluation of universally correct answers.

14.5 AI System 1 Evaluation

The first AI system's responses should be placed into a CSV containing the question, generated response, and optional reference/source fields. The file should then be uploaded to the platform. Actual scores, verdicts, dashboard metrics, and PDF outputs should be recorded directly from the completed evaluation.



Upload CSV

sample ... final.csv
3.9KB

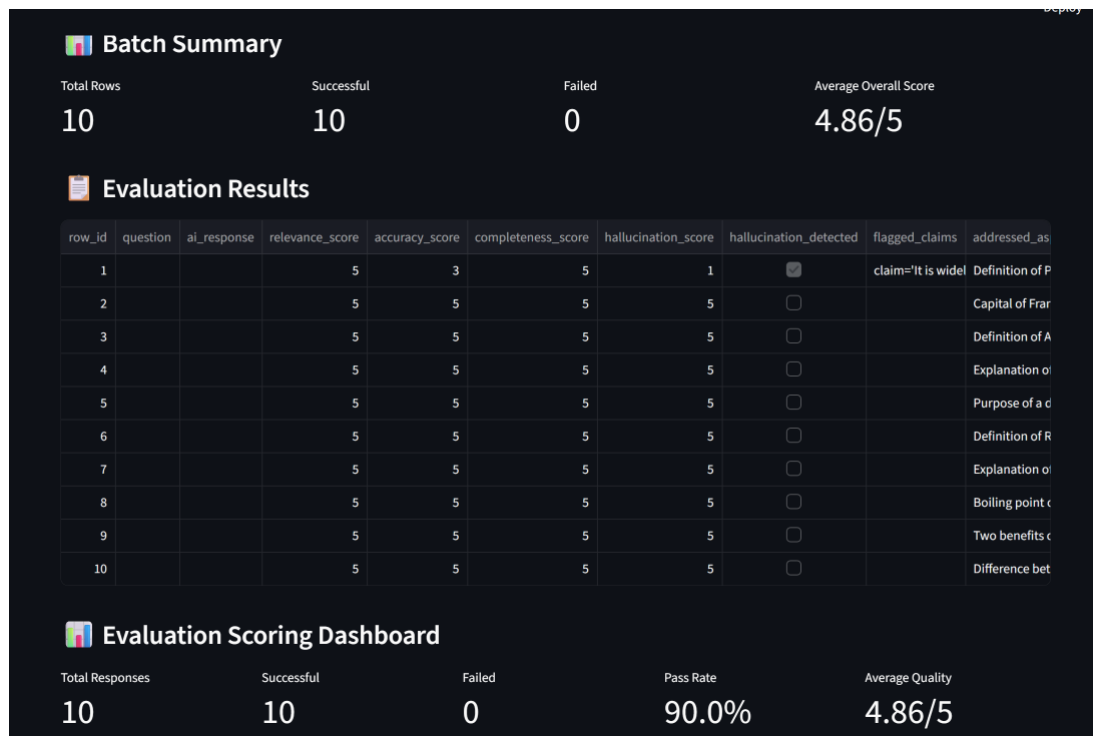
Uploaded Data

question	ai_response	reference_answer
What is Python?	Python is a high-level, interpreted programming language known for its simple and n	Python is a hi
What is the capital of France?	The capital of France is Paris.	Paris.
What is the difference between Artificial Intelligence and Machine Learning?	Artificial Intelligence (AI) is the broader concept of machines performing tasks that ne	AI is the broac
Explain supervised learning with an example.	Supervised learning is a type of machine learning where a model learns from labeled	Supervised le
What is the purpose of a database?	A database is used to store, organize, manage, and retrieve data efficiently. For exam	A database is
What is RAM and why is it important in a computer?	RAM (Random Access Memory) is the computer's temporary working memory. It store	RAM is tempo
Explain photosynthesis and mention its main inputs and outputs.	Photosynthesis is the process by which green plants use sunlight to convert carbon d	Photosynthes
What is the boiling point of water at standard atmospheric pressure?	The boiling point of water at standard atmospheric pressure (1 atm) is 100°C (212°F).	100°C (212°F)
Give two benefits and two limitations of Machine Learning.	Benefits: Automates repetitive tasks; Can identify patterns and make predictions from	Benefits: auto
What is the difference between a compiler and an interpreter?	A compiler translates the entire program into machine code before execution, while	A compiler tra

CSV structure is valid — 10 record(s) found.

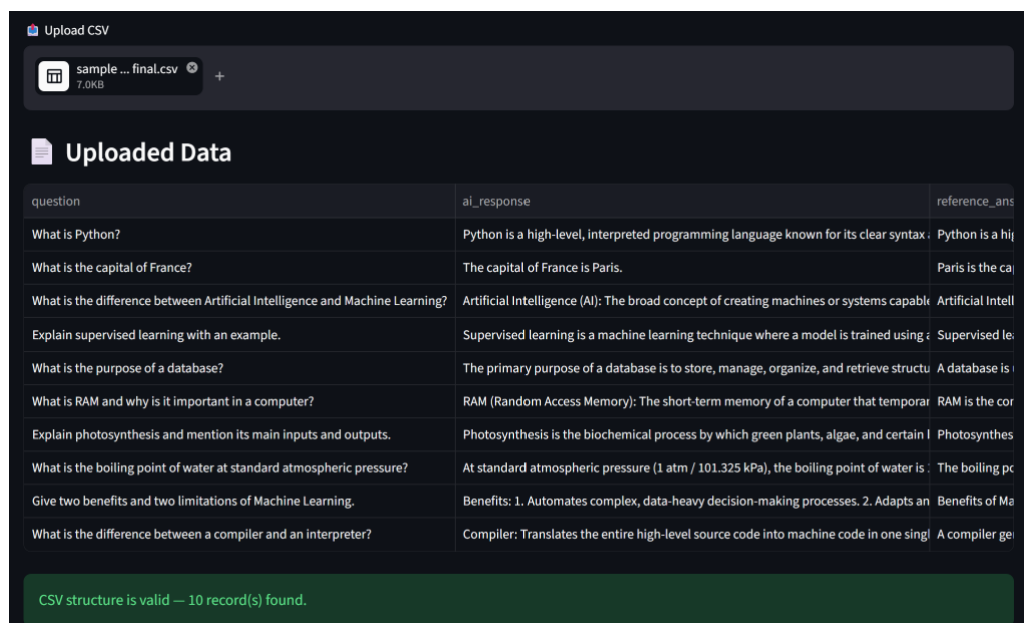
14.6 AI System 1 Results

The row-level results should record relevance, accuracy, completeness, hallucination status/score, overall score, and verdict for each question. These values must come directly from the actual platform output. No score should be manually created merely to complete the report.



14.7 AI System 2 Evaluation

The second AI system should be evaluated using the same ten questions and the same reference/source configuration. Its generated responses should be uploaded as a separate batch. The evaluation should use the same project scoring rules and model configuration used for the first system.



14.8 AI System 2 Results

The second system's row-level results should be recorded using the same fields as the first system. This ensures that the final comparison uses consistent measurements. The actual scores, verdicts, hallucination rate, and dashboard statistics should be copied from the platform after the evaluation is completed.

Batch Summary

Total Rows

10

Successful

10

Failed

0

Average Overall Score

4.67/5

Evaluation Results

row_id	question	ai_response	relevance_score	accuracy_score	completeness_score	hallucination_score	hallucination_detected	flagged_claims	addressed_as
1			5	5	5	5	☐		definition of P
2			5	5	5	5	☐		capital of Fran
3			5	5	5	5	☐		definition of A
4			5	5	5	5	☐		explanation of
5			5	4	5	1	☒	claim='structure	store data mai
6			5	4	5	1	☒	claim='It provide	definition of R
7			5	4	5	1	☒	claim='Inputs: C	explanation of
8			5	5	5	5	☐		boiling point c
9			5	5	5	5	☐		two benefits o
10			5	5	5	5	☐		definition of c

Evaluation Scoring Dashboard

Total Responses

10

Successful

10

Failed

0

Pass Rate

70.0%

Average Quality

4.67/5

14.9 Comparative Summary

The final report should compare measured metrics such as average relevance, accuracy, completeness, hallucination score, overall score, pass rate, and verdict distribution. The comparison should remain descriptive and should not claim causes for observed differences unless additional evidence supports those explanations

Batch Summary

Total Rows

10

Successful

10

Failed

0

Average Overall Score

4.86/5

Evaluation Results

row_id	question	ai_response	relevance_score	accuracy_score	completeness_score	hallucination_score	hallucination_detected	flagged_claims	addressed_as
1			5	3	5	5	☒	claim="It is wide"	Definition of P
2			5	5	5	5	☐		Capital of Frar
3			5	5	5	5	☐		Definition of A
4			5	5	5	5	☐		Explanation of
5			5	5	5	5	☐		Purpose of a c
6			5	5	5	5	☐		Definition of R
7			5	5	5	5	☐		Explanation of
8			5	5	5	5	☐		Boiling point c
9			5	5	5	5	☐		Two benefits c
10			5	5	5	5	☐		Difference bet

Evaluation Scoring Dashboard

Total Responses

10

Successful

10

Failed

0

Pass Rate

90.0%

Average Quality

4.86/5

AI response 1

Batch Summary

Total Rows

10

Successful

10

Failed

0

Average Overall Score

4.67/5

Evaluation Results

row_id	question	ai_response	relevance_score	accuracy_score	completeness_score	hallucination_score	hallucination_detected	flagged_claims	addressed_as
1			5	5	5	5	☐		definition of P
2			5	5	5	5	☐		capital of Fran
3			5	5	5	5	☐		definition of A
4			5	5	5	5	☐		explanation of
5			5	4	5	1	☒	claim="structure: store data ma	
6			5	4	5	1	☒	claim="It provide: definition of R	
7			5	4	5	1	☒	claim="Inputs: C: explanation of	
8			5	5	5	5	☐		boiling point c
9			5	5	5	5	☐		two benefits o
10			5	5	5	5	☐		definition of o

Evaluation Scoring Dashboard

Total Responses

10

Successful

10

Failed

0

Pass Rate

70.0%

Average Quality

4.67/5

AI response 2

14.11 Chapter Summary

The final demonstration validates the platform using outputs from two distinct AI systems under common evaluation conditions. Actual responses, scores, dashboard metrics, verdicts, and reports should be inserted after execution to provide authentic evidence of the system's practical operation.

CHAPTER 15

Results And Discussion

15.1 Testing Results

The implemented M4 end-to-end test suite produced 8 passing tests out of 8, with an execution time of 2.04 seconds. This represents a 100% passing rate for the executed automated suite. The result provides evidence that the tested evaluation, batch, scoring, and reporting paths behaved according to their defined test cases.

15.2 Functional Results

The project implements single evaluation, batch evaluation, deterministic verdict generation, dashboard visualization, and PDF reporting. Automated testing confirms the tested workflow behavior, while dashboard presentation and final demonstration require manual validation. The completed functionality provides an integrated path from AI response input to structured evaluation and documented output.

15.3 Interpretation of Scores

The four evaluation dimensions provide different information about response quality. A response may be highly relevant but incomplete, or relevant and complete while containing an accuracy problem. Therefore, the overall score should be interpreted together with individual dimension scores and reasoning rather than being treated as the only representation of quality.

15.4 Hallucination Results

Hallucination results should be interpreted relative to the available evidence. An unsupported claim means that the supplied context does not support the claim; it does not automatically prove that the claim is universally false. A contradicted claim has stronger evidence because the available source explicitly conflicts with the generated statement.

15.5 Dashboard Results

Dashboard metrics summarize the underlying evaluation records. Average scores, verdict distributions, and hallucination rates provide a high-level view, while detailed tables explain individual outcomes. Dashboard values should be manually verified against source evaluation records to ensure that aggregate calculations are correct.

15.6 Two-AI-System Results

The final comparison should report observed measurements from the two AI systems using the same dataset. Differences should be described through actual scores, verdict counts, flagged claims, and completeness results. Explanations about why the systems differ should only be included when supported by the evaluation evidence or additional controlled analysis.

15.7 Findings

The project demonstrates that multi-dimensional evaluation provides more information than a single score. Structured outputs simplify downstream processing, deterministic verdict rules make scoring testable, and source-based hallucination analysis provides additional transparency. Batch evaluation and dashboard reporting further support systematic examination of multiple AI-generated responses.

15.8 Chapter Summary

The results demonstrate the tested functionality of the validation platform and establish a framework for reporting the final two-AI-system demonstration. The final comparative results should be based entirely on actual project execution and supported by corresponding screenshots and exported reports.

CHAPTER 16

Limitations

16.1 External LLM Dependency

The evaluation system depends on an external Gemini API for model-based judgments. API availability, quotas, rate limits, and service conditions can affect execution. Therefore, an API-related failure should not automatically be interpreted as a failure of the application's internal evaluation logic.

16.2 Model Evaluation Variability

Model-based evaluation may produce some variation in responses despite controlled generation settings. Prompt wording, available evidence, and model behavior can influence evaluation results. Repeated testing with representative examples is therefore important for assessing consistency.

16.3 Retrieval Limitation

The current retrieval implementation uses TF-IDF rather than semantic embeddings or a vector database. While TF-IDF provides lightweight retrieval, it may be less effective when relevant information uses substantially different terminology from the query.

16.4 Evidence Coverage

Hallucination detection depends on available source information. If a relevant claim is absent from the provided source context, the system may classify it as unsupported. Therefore, hallucination results must always be interpreted in relation to the coverage and quality of the available evidence.

16.5 Historical Analytics

The current dashboard displays quality trends within the current uploaded batch. It does not provide persistent historical trends across multiple independent batches. Implementing long-term storage would be required for historical model-quality tracking.

16.6 Evaluation Ground Truth

Automated model-based evaluation should not be treated as absolute ground truth for every response. Human review or verified benchmark labels may still be required for high-stakes applications. The system should therefore be considered an evaluation-assistance platform rather than a universal replacement for expert review.

16.7 API Quota

Large or repeated evaluation workloads may encounter API quota limitations. Such limitations can occur independently of the correctness of the application code. The project should therefore consider request limits, batching strategies, retries, and alternative testing modes when performing large-scale demonstrations.

16.8 Dataset Scope

The final demonstration dataset is representative rather than exhaustive. A larger and more domain-diverse dataset would provide stronger evidence about system behavior across different topics, response styles, and failure modes.

16.9 Chapter Summary

The limitations describe important boundaries of the current system. These include dependence on external model services, retrieval limitations, evidence coverage, dataset scope, evaluation variability, and lack of persistent historical analytics. Understanding these limitations is necessary for interpreting project results correctly.

CHAPTER 17

Future Scope

17.1 Semantic Embedding Retrieval

Future versions can incorporate semantic embeddings to improve retrieval when relevant documents use different wording from the user's query. Embedding-based retrieval may provide better semantic matching than the current TF-IDF approach for more complex knowledge bases.

17.2 Vector Database Integration

A vector database could support larger document collections, efficient similarity search, metadata filtering, and persistent indexing. Such integration would improve the scalability of the retrieval layer and provide stronger infrastructure for source-grounded evaluation.

17.3 Historical Evaluation Database

Persistent storage could record previous evaluation batches, enabling historical dashboards, trend analysis, model comparisons, and long-term quality monitoring. This would extend the current batch-level dashboard into a continuous evaluation-monitoring platform.

17.4 Additional Evaluation Dimensions

Future versions could introduce additional dimensions such as clarity, citation quality, instruction following, consistency, safety, and style. Each dimension should have a clearly defined scoring methodology so that adding new metrics does not reduce interpretability.

17.5 Human-in-the-Loop Validation

A reviewer interface could allow human evaluators to confirm, modify, or reject automated evaluation results. Human feedback could then be stored as additional validation data and used to study evaluator consistency and improve evaluation prompts or scoring strategies.

17.6 Confidence and Calibration

Future research could investigate confidence estimates for automated evaluation results. Model-generated scores could be compared with verified human or benchmark labels to determine whether the evaluator's confidence corresponds to actual correctness.

17.7 Improved Hallucination Analysis

Hallucination detection could be enhanced through more detailed claim extraction and evidence alignment. Each claim could be mapped explicitly to supporting, contradicting, or absent evidence. This would improve traceability and make hallucination-related results easier to review.

17.8 Scalable Batch Processing

Future versions could implement controlled concurrency, caching, request scheduling, and quota-aware processing. These improvements would allow larger datasets to be evaluated while reducing unnecessary repeated API calls and improving overall processing efficiency.

17.9 Model Comparison

The platform could maintain evaluation records for multiple AI systems and provide controlled comparison dashboards. Such comparisons should preserve identical evaluation conditions, datasets, scoring rules, and evidence so that measured differences remain meaningful.

17.10 Deployment

The application could be deployed as a production web service with authentication, role-based access, persistent databases, monitoring, logging, and secure secret management. Such deployment would allow organizations to use the system for continuous AI response quality monitoring.

17.11 Chapter Summary

The future scope focuses on improving retrieval, scalability, persistence, human review, evaluation dimensions, hallucination analysis, model comparison, and

deployment. These enhancements could transform the current project into a more comprehensive AI quality-assurance platform.

CHAPTER 18

Conclusion

18.1 Conclusion

The AI Response Validation System with Hallucination Detection Assistance provides an integrated approach to evaluating AI-generated responses. The system examines relevance, accuracy, hallucination, and completeness through specialized agents and combines their results using deterministic weighted scoring. It also supports batch evaluation, dashboard visualization, and PDF reporting.

18.2 Final Outcome

The completed project provides a single-response evaluation workflow, four specialized evaluation dimensions, weighted verdict generation, CSV-based batch processing, an interactive dashboard, PDF report export, and automated M4 testing. The executed M4 test suite achieved 8 out of 8 passing tests in 2.04 seconds.

18.3 Final Demonstration Outcome

The final demonstration should evaluate two distinct AI systems using a common dataset. Their actual responses should be processed through the platform, and the resulting scores, verdicts, dashboard metrics, and reports should be included in the final report. This provides practical evidence that the platform can evaluate outputs from different AI systems under common conditions.

18.4 Closing Statement

The project demonstrates that AI response validation benefits from evaluating multiple dimensions rather than relying on a single quality metric. Structured outputs, evidence-based checking, deterministic aggregation, batch analytics, visualization, and reporting together provide a practical framework for

systematic AI response evaluation while maintaining awareness of the limitations of automated judgment.

References

Reference 1 — TruthfulQA

Lin, S., Hilton, J., and Evans, O. *TruthfulQA: Measuring How Models Mimic Human Falsehoods*. The work provides a benchmark for studying whether language models generate truthful answers, particularly for questions involving common misconceptions.

Reference 2 — SQuAD

Rajpurkar, P., Zhang, J., Lopyrev, K., and Liang, P. *SQuAD: 100,000+ Questions for Machine Comprehension of Text*. The dataset provides question-answering examples associated with textual passages and is relevant to evidence-based evaluation.

Reference 3 — RAG

Lewis, P. et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. The work describes combining retrieval with generation for knowledge-intensive language tasks.

Reference 4 — Gemini Documentation

Google AI for Developers documentation was consulted for Gemini API functionality and structured output integration used by the project.

Reference 5 — RAGAS

RAGAS documentation was consulted for concepts related to evaluation of Retrieval-Augmented Generation applications and multi-dimensional response assessment.

Reference 6 — TruLens

TruLens documentation was reviewed for concepts related to evaluation and monitoring of AI applications, including relevance and groundedness.

Reference 7 — Scikit-learn

Scikit-learn documentation was consulted for the TF-IDF-based text retrieval implementation used in the current project.

Reference 8 — Pydantic

Pydantic documentation was consulted for structured data models and validation used throughout the evaluation pipeline.

Reference 9 — Pandas

Pandas documentation was consulted for CSV processing, DataFrame operations, aggregation, and tabular result handling.

Reference 10 — Streamlit

Streamlit documentation was consulted for development of the interactive evaluation dashboard and application interface.

Reference 11 — ReportLab

ReportLab documentation was consulted for implementation of PDF report generation.

Reference 12 — Pytest

Pytest documentation was consulted for designing and executing automated project tests.